

Memoria técnica de implementación

AccessibleCourse

Proyecto AccessibleCourse

8 de mayo de 2026

Índice

1	Introducción y objetivo del proyecto	2
2	Arquitectura general	2
3	Tecnologías utilizadas	3
4	Flujo online con Canvas/UOC	3
5	Flujo offline con IMSCC/ZIP	4
6	Gestión segura del token de acceso	4
7	Extracción e inventario de recursos	5
8	Diagnóstico de acceso y descarga	5
9	Análisis automático de accesibilidad por tipo de recurso	6
9.1	HTML	6
9.2	PDF	6
9.3	Word/DOCX	7
9.4	Vídeo	7
9.5	Jupyter Notebook	7
10	Sistema de puntuación	8
11	Generación de informes PDF/Word/JSON	8
12	Accesibilidad de la propia interfaz	9
13	Despliegue en Railway	9
14	Pruebas y validación	10
15	Problemas encontrados y soluciones aplicadas	10
16	Limitaciones actuales	10
17	Líneas futuras de trabajo	11
18	Conclusiones	11
19	Preparación de la entrega	11

1 Introducción y objetivo del proyecto

AccessibleCourse es una aplicación full-stack orientada a revisar recursos docentes de cursos online, especialmente cursos alojados en Canvas/UOC y paquetes offline en formato IMSCC o ZIP. Su propósito técnico es construir un inventario de recursos, diagnosticar si esos recursos son accesibles o descargables, ejecutar comprobaciones automáticas de accesibilidad por tipo de recurso y producir informes reutilizables en PDF, Word y JSON.

El proyecto no pretende sustituir una auditoría humana completa. La implementación combina análisis automático, normalización de evidencias y checklist manual para que la revisión sea trazable, repetible y defendible. Esta decisión es importante porque muchos problemas de accesibilidad requieren interpretación contextual: por ejemplo, la calidad real de un texto alternativo, la adecuación pedagógica de una transcripción o el comportamiento de una herramienta LTI protegida por SSO.

La memoria se centra en la implementación existente del repositorio. Cuando una capacidad tiene limitaciones o requiere revisión humana, se indica explícitamente.

2 Arquitectura general

La aplicación está organizada en dos capas principales:

- Backend FastAPI en `app/`, responsable de API, procesamiento de cursos, análisis de accesibilidad, persistencia, seguridad, generación de informes y descarga de recursos.
- Frontend React/Vite en `src/`, responsable de la experiencia de usuario, navegación entre modos online/offline, gestión de token, listado de recursos, revisión por recurso y descarga de informes.

La estructura principal del repositorio es:

<code>app/</code>	
<code>api/routes/</code>	Rutas FastAPI agrupadas por dominio
<code>core/</code>	Configuración, errores, logging, seguridad y rate limit
<code>models/</code>	Modelos SQLAlchemy y entidades de revisión
<code>services/</code>	Lógica de negocio: Canvas, IMSCC, análisis, informes
<code>schemas/</code>	Contratos de entrada/salida
<code>src/</code>	
<code>components/</code>	Componentes reutilizables de interfaz
<code>lib/</code>	Cliente API, tipos y utilidades frontend
<code>pages/</code>	Pantallas principales
<code>styles/</code>	Estilos Tailwind y tokens visuales
<code>tests/</code>	Pruebas unitarias e integración
<code>data/</code>	Persistencia local de trabajos, subidas, extracción e informes
<code>Dockerfile</code>	Imagen combinada para despliegue tipo Railway
<code>Dockerfile.backend</code>	Imagen backend para Docker Compose
<code>Dockerfile.frontend</code>	Imagen frontend Nginx para Docker Compose
<code>docker-compose.yml</code>	Despliegue local con backend, frontend y Postgres opcional

El punto de entrada del backend está en `app/main.py`. La función `create_app()` configura logging, almacenamiento, base de datos, middleware de cabeceras de seguridad, CORS y el router principal bajo el prefijo `/api`. El servidor combinado para despliegue en una sola imagen está en `server.py`, que monta el build estático del frontend y mantiene las rutas `/api/*` para FastAPI.

Las rutas API más relevantes son:

Área	Ruta	Finalidad
Salud	GET <code>/api/health</code>	Comprobar que la API responde

Área	Ruta	Finalidad
Token	GET /api/token/status	Consultar si hay token activo o demo disponible
Token	POST /api/token/configure	Validar y configurar token de usuario
Token	POST /api/token/activate-demo	Activar token demo del backend si existe
Token	POST /api/token/deactivate	Eliminar sesión de token
Canvas	GET /api/canvas/courses	Listar cursos accesibles con el token
Canvas	POST /api/canvas/jobs	Crear análisis online de un curso Canvas
Offline	POST /api/jobs	Subir IMSCC/ZIP y crear análisis offline
Jobs	GET /api/jobs/{job_id}	Consultar estado del procesamiento
Recursos	GET /api/jobs/{job_id}/resources	Obtener inventario normalizado
Recursos	GET /api/jobs/{job_id}/resources/{resource_id}	Detalle y checklist de un recurso
Contenido	GET /api/jobs/{job_id}/resources/{resource_id}/content	Recuperar contenido reutilizable
Acceso	GET /api/jobs/{job_id}/access	Diagnóstico de acceso por recurso
Accesibilidad	GET /api/jobs/{job_id}/accessibility	Resultados automáticos por tipo
Reintento	POST /api/jobs/{job_id}/access-analysis/retry	Reejecutar diagnóstico de acceso
Informes	POST /api/jobs/{job_id}/report	Generar informe técnico
Informes	GET /api/jobs/{job_id}/report/download?format=pdf	Descargar PDF
Informes	GET /api/jobs/{job_id}/report/download?format=docx	Descargar Word
Informes	GET /api/jobs/{job_id}/report/download?format=json	Descargar JSON

3 Tecnologías utilizadas

En backend se usan FastAPI, Starlette, SQLAlchemy, Pydantic Settings, HTTPX, python-multipart, pypdf, python-docx, ReportLab, BeautifulSoup y cryptography. La base de datos por defecto es SQLite (DATABASE_URL), con soporte de Postgres mediante la configuración de entorno y el servicio opcional db de docker-compose.yml.

En frontend se usan React 18, React Router, TypeScript, Vite, Tailwind CSS y ESLint. El cliente API se centraliza en `src/lib/api.ts`, donde se normalizan respuestas de backend para mantener una interfaz estable aunque existan campos heredados o nombres equivalentes.

La generación de informes se implementa en `app/services/reports.py`. El Word se genera con `python-docx`, el PDF con ReportLab o mediante conversión de DOCX si existe LibreOffice/soffice, y el JSON se persiste como payload estructurado.

4 Flujo online con Canvas/UOC

El flujo online comienza con la gestión del token. El frontend consulta `GET /api/token/status` desde `src/App.tsx` y, si no hay token activo, muestra las pantallas de bienvenida o configuración

(`src/pages/TokenWelcomePage.tsx`, `src/pages/TokenConfigurePage.tsx` y `src/pages/TokenManagementPage.tsx`).

Una vez configurado el token, el frontend llama a `GET /api/canvas/courses`, implementado en `app/api/routes/canvas.py`. El backend utiliza `CanvasAPIClient` (`app/services/canvas_api.py`) y `CanvasClient` (`app/services/canvas_client.py`) para comunicarse con Canvas mediante la API `/api/v1`. Las peticiones usan cabecera `Authorization: Bearer ...`, pero el valor del token no se expone en logs ni en documentación.

Cuando el usuario selecciona un curso, `POST /api/canvas/jobs` crea un job online. La función `process_online_job()` de `app/services/jobs.py` valida autenticación, obtiene curso, módulos e ítems, construye el inventario inicial con `build_canvas_inventory()` (`app/services/canvas_inventory.py`) y ejecuta el diagnóstico de acceso con `analyze_access()` y `OnlineAccessAdapter` (`app/services/access_analysis.py`).

El flujo online distingue entre:

- Ficheros Canvas, que pueden disponer de URL de descarga y metadatos de tipo MIME.
- Páginas Canvas, tareas, discusiones y quizzes, que pueden contener HTML recuperable mediante API.
- Enlaces externos, cuyo acceso se comprueba mediante `URLCheckService`.
- Herramientas externas, LTI, RALTI o recursos que redirigen a SSO, que se marcan como no analizables automáticamente o como recursos que requieren interacción.

El procesamiento online no automatiza una sesión de navegador completa. Esto evita depender de flujos SSO frágiles y reduce el riesgo de manejar credenciales fuera del backend controlado, pero obliga a marcar algunos recursos como revisión manual.

5 Flujo offline con IMSCC/ZIP

El flujo offline se inicia en `POST /api/jobs`, definido en `app/api/routes/jobs.py`. El backend valida la extensión del archivo (`.imsc` o `.zip`), guarda la subida con `save_upload_file()` y comprueba que el archivo sea realmente un ZIP. La configuración de tamaño máximo se controla con `MAX_UPLOAD_MB`.

El procesamiento se ejecuta en segundo plano mediante `process_job()` (`app/services/jobs.py`). Sus fases principales son:

1. Validación de archivo y creación de job.
2. Extracción segura del ZIP en `data/jobs/<job_id>/extracted`.
3. Localización y parseo de `imsmanifest.xml`.
4. Reconstrucción de estructura de curso e inventario.
5. Diagnóstico de acceso y deep scan de recursos enlazados.
6. Ejecución de analizadores automáticos de accesibilidad.
7. Persistencia de inventario, estructura y resultados.

La extracción segura se implementa en `app/services/storage.py`. Se bloquean rutas absolutas, rutas con `..`, enlaces simbólicos dentro del ZIP y archivos que exceden los límites de cantidad o tamaño (`MAX_EXTRACTED_FILES` y `MAX_EXTRACTED_MB`). Esta medida protege frente a problemas habituales de paquetes comprimidos, como path traversal o ZIPs excesivamente grandes.

El parser IMSCC principal está en `app/services/imsc_parser.py`. Además de leer el manifiesto, conserva la estructura de módulos, identifica recursos internos y externos, clasifica tipos por extensión o URL, y descubre recursos enlazados desde HTML local. Si el manifiesto no permite reconstruir el curso, existe una ruta de fallback en `app/services/imsc.py`.

6 Gestión segura del token de acceso

La gestión de token está separada del inventario y del análisis. Las rutas están en `app/api/routes/token.py` y la lógica de sesión en `app/services/token_session.py`.

El token de usuario se valida contra Canvas antes de aceptarse. Si la validación falla, el backend responde con error y no activa la sesión. Si la validación es correcta, el token se cifra con Fernet usando `TOKEN_ENCRYPTION_KEY` y se guarda asociado a una sesión temporal en `data/sessions`. La cookie enviada al navegador contiene un identificador de sesión firmado con HMAC, no el token de Canvas.

La cookie de sesión se configura como `HttpOnly`; en producción también se fuerza `Secure` mediante la función `_cookie_secure()`. El modo demo, si existe, usa el token configurado en el backend con `CANVAS_TOKEN`, pero la memoria y el repositorio no incluyen valores reales de tokens.

Además, `record_job_event()` en `app/services/jobs.py` aplica redacción de claves sensibles en los detalles de log. Los nombres de variables sensibles pueden aparecer en documentación o ejemplos, pero no deben incluir valores reales.

7 Extracción e inventario de recursos

El modelo técnico común de un recurso se normaliza en `app/services/resource_core.py` mediante `ResourceCore`. Esta capa evita que cada flujo trabaje con estructuras incompatibles. Entre los campos normalizados están:

- `id`, `title`, `type` y `origin`.
- `modulePath` y `sectionTitle`.
- `accessStatus`, `reasonCode`, `reasonDetail` y `httpStatus`.
- `downloadable`, `downloadStatus`, `localPath`, `htmlPath` y `sourceUrl`.
- `contentAvailable`, que indica si el backend puede reutilizar el contenido para análisis.

La clasificación de tipos cubre HTML/Web, PDF, DOCX, vídeo, imagen, notebook, archivo genérico y otros. En el caso offline, `IMSCCParser` identifica ficheros locales, URLs externas y recursos descubiertos dentro de páginas HTML. En el caso online, `build_canvas_inventory()` transforma módulos e ítems de Canvas en recursos homogéneos, conservando metadatos como `canvasType`, `courseId`, `moduleId`, `fileId`, `downloadUrl` o `pageUrl`.

El inventario persistido se guarda en `data/jobs/<job_id>/resources.json` y se sincroniza también con las tablas `SQLModel` (`resources`, `checklist_templates`, `checklist_responses`, `review_summaries`). Esto permite combinar análisis automático, revisión manual y generación de informes sin depender únicamente de archivos JSON.

8 Diagnóstico de acceso y descarga

El diagnóstico de acceso se implementa principalmente en `app/services/access_analysis.py`, con adaptadores separados para recursos offline (`OfflineAccessAdapter`) y online (`OnlineAccessAdapter`). El objetivo es determinar no solo si un recurso existe, sino también si es reutilizable para análisis o descarga.

Los estados principales son:

- `OK`: el recurso es accesible o existe localmente.
- `NO_ACCEDE`: el recurso no se puede acceder por error HTTP, red, permiso o ausencia de archivo.
- `REQUIERE_SSO`: el recurso redirige a autenticación externa o depende de una capa SSO.
- `REQUIERE_INTERACCION`: el recurso requiere una sesión interactiva o acción humana.
- `NO_ANALIZABLE`: el recurso queda fuera del análisis automático actual.

Para URLs externas, `URLCheckService` (`app/services/url_check.py`) intenta primero `HEAD` y cae a `GET` si el servidor no acepta `HEAD`. Registra código HTTP, URL final, redirecciones, tipo de contenido y motivo de fallo. Para Canvas, si la URL pertenece al mismo host, las comprobaciones pueden incorporar credenciales del token activo.

La descarga se expone por `GET /api/jobs/{job_id}/resources/{resource_id}/download`. En recursos locales se resuelve la ruta mediante `resolve_job_resource_path()` para impedir que una ruta salga del directorio extraído. En recursos Canvas descargables, el backend puede cachear el fichero bajo `data/jobs/<job_id>/online_downloads`.

9 Análisis automático de accesibilidad por tipo de recurso

Los analizadores automáticos comparten el modelo `AccessibilityCheckResult` definido en `app/services/html_accessibility.py`. Cada check devuelve identificador, título, estado, evidencia, recomendación y, cuando aplica, una referencia WCAG orientativa. Los estados son `PASS`, `FAIL`, `WARNING`, `NOT_APPLICABLE` y `ERROR`.

9.1 HTML

El análisis HTML está en `app/services/html_accessibility.py`. Se ejecuta sobre recursos WEB con contenido disponible y que no requieren SSO o interacción. El backend recupera HTML local o HTML de Canvas mediante `get_resource_content()`.

Los checks implementados revisan:

- Idioma principal en `<html lang>`.
- Título de página.
- Existencia de un encabezado principal `h1`.
- Saltos en la jerarquía de encabezados.
- Imágenes con `alt`.
- Textos de enlace descriptivos.
- Botones con nombre accesible.
- Campos de formulario con etiqueta.
- Iframes con título.
- Tablas con encabezados, `caption` o asociaciones.

No se ejecuta un navegador ni se calculan estilos CSS finales. Por tanto, criterios como contraste real, navegación por teclado o foco visible quedan cubiertos principalmente por checklist manual y revisión complementaria.

9.2 PDF

El análisis PDF está en `app/services/pdf_accessibility.py` y utiliza `pypdf`. Se ejecuta sobre PDFs accesibles como fichero local o descargado/cacheado.

Los checks implementados revisan:

- Apertura del documento y ausencia de cifrado.
- Existencia de texto extraíble.
- Idioma del documento en `/Lang`.
- Título en metadatos.
- PDF etiquetado mediante `/MarkInfo` y `/StructTreeRoot`.
- Encabezados estructurados en el árbol de etiquetas.
- Figuras con `/Alt`.
- Tablas estructuradas.
- Anotaciones de enlace con destino.
- Marcadores en documentos largos.

El sistema detecta indicios de PDF escaneado cuando no puede extraer texto significativo, pero no aplica OCR automáticamente. La corrección de OCR queda como acción futura o manual.

9.3 Word/DOCX

El análisis Word está en `app/services/docx_accessibility.py`. Usa `python-docx` y lectura directa del XML interno del paquete DOCX.

Los checks implementados revisan:

- Documento legible y parseable.
- Texto extraíble.
- Idioma declarado en estilos, configuración o propiedades.
- Título en propiedades del documento.
- Uso de estilos de encabezado.
- Jerarquía de encabezados.
- Imágenes con texto alternativo.
- Tablas con encabezados identificables.
- Enlaces con texto descriptivo.
- Listas estructuradas frente a listas manuales.

Actualmente, la plantilla de checklist manual de DOCX reutiliza la categoría genérica `OTHER` en `app/services/template_seed.py`, aunque el análisis automático específico de DOCX sí está implementado.

9.4 Vídeo

El análisis de vídeo está en `app/services/video_accessibility.py`. La implementación es deliberadamente conservadora: no descarga vídeos de plataformas externas por defecto. Para vídeos locales o descargables puede inspeccionar metadatos preliminares; para embeds o enlaces externos usa señales HTML y metadatos del recurso.

Los checks implementados revisan:

- Accesibilidad general del recurso según diagnóstico previo.
- Título descriptivo.
- Proveedor detectado.
- Necesidad de revisión manual del proveedor.
- Iframe con título.
- Subtítulos o pistas detectables.
- Transcripción disponible.
- Señales de audiodescripción o alternativa textual.
- Controles de reproducción.
- Autoplay.
- Metadatos locales cuando hay fichero disponible.

La revisión completa del contenido audiovisual sigue requiriendo validación humana, especialmente para calidad de subtítulos, sincronización, identificación de hablantes y suficiencia de la descripción visual.

9.5 Jupyter Notebook

El análisis de Jupyter Notebook está implementado en `app/services/notebook_accessibility.py`. Se trata de un análisis estático de ficheros `.ipynb`; no ejecuta código, no instala kernels y no modifica el notebook.

Los checks implementados revisan:

- Notebook legible y parseable como JSON.
- Existencia de texto explicativo inicial en Markdown.
- Título principal en Markdown.

- Jerarquía de encabezados Markdown.
- Proporción y utilidad de celdas Markdown explicativas.
- Imágenes Markdown con texto alternativo.
- Enlaces descriptivos y URLs desnudas.
- Salidas visuales guardadas.
- Errores de ejecución almacenados.
- Orden de ejecución.
- Tablas Markdown con estructura reconocible.

Esta funcionalidad está operativa, pero con la limitación propia de un análisis estático: no puede garantizar que el notebook ejecute correctamente en un entorno real ni validar dependencias externas.

10 Sistema de puntuación

El sistema de puntuación se construye durante la generación de informes en `app/services/reports.py`. Para cada recurso analizado automáticamente se consideran los checks aplicables:

- PASS suma 1 punto.
- WARNING suma 0,5 puntos.
- FAIL y ERROR suman 0 puntos.
- NOT_APPLICABLE no penaliza.

El score por recurso se calcula como porcentaje sobre los checks aplicables. Si no hay checks aplicables, el recurso no aporta incidencias automáticas relevantes y se trata de forma conservadora en el informe.

La prioridad se deriva del score y de checks configurados como críticos o importantes:

- Prioridad alta: score inferior a 60 o fallo crítico.
- Prioridad media: score inferior a 80 o warning importante.
- Prioridad baja: score igual o superior a 80 sin señales críticas.

El score por módulo se calcula agregando los scores de los recursos analizados dentro del módulo o sección. El score global del informe se calcula a partir de los recursos con análisis automático. Cuando no hay recursos analizados pero sí recursos detectados, el score global se mantiene en 0 para evitar una interpretación positiva sin evidencia.

El informe incluye:

- Score global en el resumen ejecutivo.
- Score por módulo o sección.
- Score por recurso.
- Prioridad global, por módulo y por recurso.
- Incidencia principal de cada recurso.
- Recomendaciones priorizadas a partir de los checks con FAIL o WARNING.

11 Generación de informes PDF/Word/JSON

La generación canónica de informes se expone en `POST /api/jobs/{job_id}/report`. El payload se construye con `_build_report_payload()` en `app/services/reports.py`, que integra inventario, diagnóstico de acceso, resultados automáticos, revisión manual, scores y recursos no analizables automáticamente.

Los archivos se persisten en:

```
data/jobs/<job_id>/report/report.json
data/jobs/<job_id>/report/report.docx
data/jobs/<job_id>/report/report.pdf
```

El informe Word se genera con `python-docx`; el PDF se genera con ReportLab o por conversión de DOCX si el entorno tiene LibreOffice/soffice. El JSON conserva la estructura completa para reutilización o auditoría posterior.

El backend también mantiene rutas heredadas bajo `/api/reports/{job_id}` para compatibilidad, pero el frontend actual usa las descargas directas de `/api/jobs/{job_id}/report/download?format=...`, definidas en `src/lib/api.ts`.

Las descargas añaden cabeceras `Cache-Control: no-store`, `Pragma: no-cache` y `Expires: 0` para reducir el riesgo de conservar informes con información sensible en cachés intermedias.

12 Accesibilidad de la propia interfaz

La interfaz se construye con React y Tailwind CSS. El componente `LayoutSimple` (`src/components/LayoutSimple.tsx`) incorpora varias decisiones de accesibilidad:

- Enlace de salto al contenido.
- Uso de landmark principal cuando corresponde.
- Foco programático en el encabezado al cambiar de pantalla.
- Botones reales para acciones.
- Estados de error con `role="alert"` y `aria-live`.
- Formularios con etiquetas visibles.
- Fieldsets y radios para checklists.
- Indicadores de foco visibles definidos en `src/styles/index.css`.

El flujo de token evita mostrar información sensible de forma persistente en el navegador. En `TokenConfigurePage.tsx` el campo usa `type="password"` por defecto y permite mostrar/ocultar el valor solo durante la entrada.

La interfaz prioriza claridad operativa: subida offline, selección online, estado del análisis, revisión de recursos y descarga del informe. No obstante, no se ha observado en el repositorio una prueba automatizada de accesibilidad frontend con herramientas como axe-core o Playwright; esto queda como línea futura.

13 Despliegue en Railway

El repositorio incluye un `Dockerfile` pensado para despliegues en una sola imagen. La primera etapa compila el frontend con Node 20 y Vite; la segunda instala dependencias Python, copia `app/`, `server.py` y el build del frontend en `public/`. El comando final arranca:

```
uvicorn server:app --host 0.0.0.0 --port ${PORT:-10000}
```

Esta forma encaja con Railway porque Railway suele inyectar la variable `PORT`. `server.py` sirve la SPA y delega `/api/*` al backend FastAPI en el mismo proceso.

Para despliegue local o VPS, `docker-compose.yml` separa backend y frontend. El backend usa `Dockerfile.backend`; el frontend usa `Dockerfile.frontend` con Nginx y proxy `/api/` hacia el backend. También se define un servicio Postgres opcional mediante perfil `postgres`.

En Railway deben configurarse variables de entorno sin incluir secretos en el repositorio. Entre ellas: `DATABASE_URL`, `STORAGE_ROOT`, `CORS_ORIGINS`, `CANVAS_BASE_URL`, `CANVAS_API_PREFIX`, `TOKEN_ENCRYPTION_KEY`, `SESSION_SECRET`, `TOKEN_COOKIE_SECURE` y, solo si se quiere modo demo, `CANVAS_TOKEN`.

Una limitación práctica del despliegue en Railway es la persistencia. El proyecto guarda subidas, recursos extraídos, sesiones cifradas e informes bajo `STORAGE_ROOT`. Para producción conviene usar volumen persistente o adaptar el almacenamiento a un servicio externo si se necesitan informes históricos o sesiones duraderas.

14 Pruebas y validación

El repositorio incluye pruebas en `tests/` para API, cliente Canvas, clasificación de estados online, deep scan, parser/inventario, análisis HTML, PDF, DOCX, vídeo, notebook, URL check, resumen ejecutivo y rutas de despliegue.

Se ejecutaron las siguientes validaciones en el entorno local del repositorio:

```
.venv/bin/ruff check app tests
.venv/bin/pytest
npm run lint
```

Resultados obtenidos:

- **ruff**: correcto, sin errores.
- **pytest**: 97 tests superados; aparece una advertencia de deprecación de Starlette en `tests/test_deploy_routes.py`.
- **npm run lint**: correcto, sin errores de ESLint. npm mostró una advertencia experimental de Node sobre carga CommonJS/ESM, no relacionada con el código del proyecto.

Estas pruebas dan cobertura a la lógica crítica de backend y a la calidad estática del frontend. No sustituyen pruebas manuales con un curso Canvas real ni validación visual completa de informes con datos reales.

15 Problemas encontrados y soluciones aplicadas

Durante la implementación se abordan varios problemas técnicos típicos de este tipo de herramienta:

- Paquetes comprimidos inseguros o muy grandes. Se aplican límites de tamaño, número de ficheros, rutas permitidas y bloqueo de enlaces simbólicos en `app/services/storage.py`.
- IMSCC con manifiestos incompletos o estructuras no homogéneas. Se implementa parser específico en `app/services/imsc_parser.py` y fallback por exploración de ficheros en `app/services/imsc.py`.
- Recursos externos protegidos por SSO o LTI. Se clasifican como `REQUIERE_SSO` o `REQUIERE_INTERACCION` para evitar falsos positivos y orientar revisión manual.
- URLs que no aceptan `HEAD`. `URLCheckService` cae a `GET` cuando corresponde.
- Diferencias de modelo entre offline y Canvas. `ResourceCore` normaliza campos y estados para que los analizadores trabajen sobre un contrato común.
- Riesgo de exposición de credenciales. El token se cifra, la cookie solo contiene un identificador firmado y los logs redactan claves sensibles.
- Informes demasiado extensos. El PDF y DOCX resumen incidencias y omiten checks `PASS` en el detalle técnico para mantener el informe accionable.

16 Limitaciones actuales

Las principales limitaciones actuales son:

- El análisis HTML es estático y no evalúa contraste CSS real, foco visible ni navegación por teclado en navegador.

- Los recursos protegidos por SSO, RALTI o LTI requieren revisión manual.
- El análisis PDF detecta ausencia de texto extraíble, pero no aplica OCR.
- El análisis de vídeo no verifica automáticamente la calidad real de subtítulos, transcripción o audiodescripción.
- El análisis de notebooks no ejecuta código ni valida dependencias.
- La persistencia por filesystem requiere atención en despliegues cloud con almacenamiento efímero.
- La interfaz no tiene todavía pruebas automatizadas de accesibilidad frontend con navegador real.
- La configuración de tokens depende de variables de entorno seguras y de una clave de cifrado correctamente gestionada.

17 Líneas futuras de trabajo

Como evolución técnica, se proponen las siguientes líneas:

- Integrar auditoría frontend con axe-core y pruebas de teclado mediante Playwright.
- Añadir análisis visual de contraste real para HTML renderizado.
- Incorporar OCR opcional para PDFs escaneados.
- Mejorar análisis de vídeo con integración con proveedores que expongan subtítulos o transcripciones por API.
- Añadir cola de trabajos persistente para procesamientos largos en producción.
- Migrar almacenamiento de informes y recursos a object storage en despliegues cloud.
- Incorporar OAuth institucional o LTI Advantage en lugar de token personal cuando el entorno lo permita.
- Calibrar el sistema de scoring con rúbricas académicas y muestras reales.
- Añadir exportación de evidencias para revisión manual y trazabilidad de correcciones.

18 Conclusiones

AccessibleCourse implementa una arquitectura coherente para analizar recursos docentes desde dos fuentes: paquetes offline IMSCC/ZIP y cursos online Canvas/UOC. La solución separa inventario, diagnóstico de acceso, análisis automático, revisión manual e informes, lo que permite defender técnicamente sus resultados y explicar sus límites.

La decisión de marcar recursos protegidos o interactivos como no analizables automáticamente es adecuada para un entorno académico real, donde forzar accesos o simular sesiones SSO puede ser inseguro o poco estable. Del mismo modo, combinar checks automáticos con checklist manual evita presentar el sistema como una auditoría absoluta.

El estado actual del proyecto es funcional para generar inventarios, detectar barreras frecuentes en HTML, PDF, DOCX, vídeo y notebooks, y producir informes accionables. Las mejoras futuras deberían centrarse en navegador real, persistencia cloud, OCR, integración institucional y validación con cursos reales.

19 Preparación de la entrega

Para crear un ZIP limpio del repositorio desde el commit actual, se recomienda usar:

```
git archive --format=zip --output accessiblecourse_entrega.zip HEAD
```

Este comando empaqueta únicamente el contenido versionado en HEAD. No incluye archivos sin trackear, directorios de ejecución como `data/`, ni valores locales de entorno.